Chapter 10. Joins Revisited

By now, you should be comfortable with the concept of the inner join, which I introduced in Chapter 5. This chapter focuses on other ways in which you can join tables, including the outer join and the cross join.

Outer Joins

In all the examples thus far that have included multiple tables, we haven't been concerned that the join conditions might fail to find matches for all the rows in the tables. For example, the `inventory` table contains a row for every film available for rental, but of the 1,000 rows in the `film` table, only 958 have one or more rows in the `inventory` table. The other 42 films are not available for rental (perhaps they are new releases due to arrive in a few days), so these film IDs cannot be found in the `inventory` table. The following query counts the number of available copies of each film by joining these two tables:

```
mysql> SELECT f.film_id, f.title, count(*) num_copies
    -> FROM film f
    ->   INNER JOIN inventory i
    ->   ON f.film_id = i.film_id
    -> GROUP BY f.film_id, f.title;
+---------+----------------------------+------------+
| film_id | title                      | num_copies |
+---------+----------------------------+------------+
|       1 | ACADEMY DINOSAUR           |          8 |
|       2 | ACE GOLDFINGER             |          3 |
|       3 | ADAPTATION HOLES           |          4 |
|       4 | AFFAIR PREJUDICE           |          7 |
...
|      13 | ALI FOREVER                |          4 |
|      15 | ALIEN CENTER               |          6 |
...
|     997 | YOUTH KICK                 |          2 |
|     998 | ZHIVAGO CORE               |          2 |
|     999 | ZOOLANDER FICTION          |          5 |
```

```
|   1000 | ZORRO ARK                |         8 |
+--------+--------------------------+-----------+
958 rows in set (0.02 sec)
```

While you may have expected 1,000 rows to be returned (one for each film), the query returns only 958 rows. This is because the query uses an inner join, which only returns rows that satisfy the join condition. The film *Alice Fantasia* (`film_id 14`) doesn't appear in the results, for example, because it doesn't have any rows in the `inventory` table.

If you want the query to return all 1,000 films, regardless of whether or not there are rows in the `inventory` table, you can use an `outer` join, which essentially makes the join condition optional:

```
mysql> SELECT f.film_id, f.title, count(i.inventory_id) num_copies
    -> FROM film f
    ->   LEFT OUTER JOIN inventory i
    ->   ON f.film_id = i.film_id
    -> GROUP BY f.film_id, f.title;
+---------+----------------------------+------------+
| film_id | title                      | num_copies |
+---------+----------------------------+------------+
|       1 | ACADEMY DINOSAUR           |          8 |
|       2 | ACE GOLDFINGER             |          3 |
|       3 | ADAPTATION HOLES           |          4 |
|       4 | AFFAIR PREJUDICE           |          7 |
...
|      13 | ALI FOREVER                |          4 |
|      14 | ALICE FANTASIA             |          0 |
|      15 | ALIEN CENTER               |          6 |
...
|     997 | YOUTH KICK                 |          2 |
|     998 | ZHIVAGO CORE               |          2 |
|     999 | ZOOLANDER FICTION          |          5 |
|    1000 | ZORRO ARK                  |          8 |
+---------+----------------------------+------------+
1000 rows in set (0.01 sec)
```

As you can see, the query now returns all 1,000 rows from the `film` table, and 42 of the rows (including *Alice Fantasia*) have a value of 0 in the `num_copies` column, which indicates that there are no copies in inventory.

Here's a description of the changes from the prior version of the query:

- The join definition was changed from `inner` to `left outer`, which instructs the server to include all rows from the table on the left side of the join (`film`, in this case) and then include columns from the table on the right side of the join (`inventory`) if the join is successful.
- The `num_copies` column definition was changed from `count(*)` to `count(i.inventory_id)`, which will count the number of non-`null` values of the `inventory.inventory_id` column.

Next, let's remove the `group by` clause and filter out most of the rows in order to clearly see the differences between `inner` and `outer` joins. Here's a query using an `inner` join and a filter condition to return rows for just a few films:

```
mysql> SELECT f.film_id, f.title, i.inventory_id
    -> FROM film f
    ->   INNER JOIN inventory i
    ->   ON f.film_id = i.film_id
    -> WHERE f.film_id BETWEEN 13 AND 15;
+---------+--------------+--------------+
| film_id | title        | inventory_id |
+---------+--------------+--------------+
|      13 | ALI FOREVER  |           67 |
|      13 | ALI FOREVER  |           68 |
|      13 | ALI FOREVER  |           69 |
|      13 | ALI FOREVER  |           70 |
|      15 | ALIEN CENTER |           71 |
|      15 | ALIEN CENTER |           72 |
|      15 | ALIEN CENTER |           73 |
|      15 | ALIEN CENTER |           74 |
|      15 | ALIEN CENTER |           75 |
|      15 | ALIEN CENTER |           76 |
+---------+--------------+--------------+
10 rows in set (0.00 sec)
```

The results show that there are four copies of *Ali Forever* and six copies of *Alien Center* in inventory. Here's the same query, but using an `outer` join:

```
mysql> SELECT f.film_id, f.title, i.inventory_id
    -> FROM film f
    ->   LEFT OUTER JOIN inventory i
    ->   ON f.film_id = i.film_id
    -> WHERE f.film_id BETWEEN 13 AND 15;
+---------+---------------+--------------+
```

```
| film_id | title         | inventory_id |
+---------+---------------+--------------+
|      13 | ALI FOREVER   |           67 |
|      13 | ALI FOREVER   |           68 |
|      13 | ALI FOREVER   |           69 |
|      13 | ALI FOREVER   |           70 |
|      14 | ALICE FANTASIA |         NULL |
|      15 | ALIEN CENTER  |           71 |
|      15 | ALIEN CENTER  |           72 |
|      15 | ALIEN CENTER  |           73 |
|      15 | ALIEN CENTER  |           74 |
|      15 | ALIEN CENTER  |           75 |
|      15 | ALIEN CENTER  |           76 |
+---------+---------------+--------------+
11 rows in set (0.00 sec)
```

The results are the same for *Ali Forever* and *Alien Center*, but there's one new row for *Alice Fantasia*, with a `null` value for the `inventory.inventory_id` column. This example illustrates how an `outer` join will add column values without restricting the number of rows returned by the query. If the join condition fails (as in the case of *Alice Fantasia*), any columns retrieved from the outer-joined table will be `null`.

Left Versus Right Outer Joins

In each of the outer join examples in the previous section, I specified `left outer join`. The keyword `left` indicates that the table on the left side of the join is responsible for determining the number of rows in the result set, whereas the table on the right side is used to provide column values whenever a match is found. However, you may also specify a `right outer join`, in which case the table on the right side of the join is responsible for determining the number of rows in the result set, whereas the table on the left side is used to provide column values.

Here's the last query from the previous section rearranged to use a `right outer join` instead of a `left outer join`:

```
mysql> SELECT f.film_id, f.title, i.inventory_id
    -> FROM inventory i
    ->   RIGHT OUTER JOIN film f
    ->   ON f.film_id = i.film_id
    -> WHERE f.film_id BETWEEN 13 AND 15;
+---------+---------------+--------------+
| film_id | title         | inventory_id |
```

```
+---------+---------------+-------------+
|     13 | ALI FOREVER    |         67 |
|     13 | ALI FOREVER    |         68 |
|     13 | ALI FOREVER    |         69 |
|     13 | ALI FOREVER    |         70 |
|     14 | ALICE FANTASIA |       NULL |
|     15 | ALIEN CENTER   |         71 |
|     15 | ALIEN CENTER   |         72 |
|     15 | ALIEN CENTER   |         73 |
|     15 | ALIEN CENTER   |         74 |
|     15 | ALIEN CENTER   |         75 |
|     15 | ALIEN CENTER   |         76 |
+---------+---------------+-------------+
11 rows in set (0.00 sec)
```

Keep in mind that both versions of the query are performing outer joins; the keywords `left` and `right` are there just to tell the server which table is allowed to have gaps in the data. If you want to outer-join tables A and B and you want all rows from A with additional columns from B whenever there is matching data, you can specify either `A left outer join B` or `B right outer join A`.

NOTE

Since you will rarely (if ever) encounter right outer joins, and since not all database servers support them, I recommend that you always use left outer joins. The `outer` keyword is optional, so you may opt for `A left join B` instead, but I recommend including `outer` for the sake of clarity.

Three-Way Outer Joins

In some cases, you may want to outer-join one table with two other tables. For example, the query from a prior section can be expanded to include data from the `rental` table:

```
mysql> SELECT f.film_id, f.title, i.inventory_id, r.rental_date
    -> FROM film f
    ->   LEFT OUTER JOIN inventory i
    ->   ON f.film_id = i.film_id
    ->   LEFT OUTER JOIN rental r
    ->   ON i.inventory_id = r.inventory_id
    -> WHERE f.film_id BETWEEN 13 AND 15;
+---------+---------------+-------------+---------------------+
| film_id | title         | inventory_id | rental_date         |
+---------+---------------+-------------+---------------------+
```

```
|     13 | ALI FOREVER    |         67 | 2005-07-31 18:11:17 |
|     13 | ALI FOREVER    |         67 | 2005-08-22 21:59:29 |
|     13 | ALI FOREVER    |         68 | 2005-07-28 15:26:20 |
|     13 | ALI FOREVER    |         68 | 2005-08-23 05:02:31 |
|     13 | ALI FOREVER    |         69 | 2005-08-01 23:36:10 |
|     13 | ALI FOREVER    |         69 | 2005-08-22 02:12:44 |
|     13 | ALI FOREVER    |         70 | 2005-07-12 10:51:09 |
|     13 | ALI FOREVER    |         70 | 2005-07-29 01:29:51 |
|     13 | ALI FOREVER    |         70 | 2006-02-14 15:16:03 |
|     14 | ALICE FANTASIA |       NULL | NULL                |
|     15 | ALIEN CENTER   |         71 | 2005-05-28 02:06:37 |
|     15 | ALIEN CENTER   |         71 | 2005-06-17 16:40:03 |
|     15 | ALIEN CENTER   |         71 | 2005-07-11 05:47:08 |
|     15 | ALIEN CENTER   |         71 | 2005-08-02 13:58:55 |
|     15 | ALIEN CENTER   |         71 | 2005-08-23 05:13:09 |
|     15 | ALIEN CENTER   |         72 | 2005-05-27 22:49:27 |
|     15 | ALIEN CENTER   |         72 | 2005-06-19 13:29:28 |
|     15 | ALIEN CENTER   |         72 | 2005-07-07 23:05:53 |
|     15 | ALIEN CENTER   |         72 | 2005-08-01 05:55:13 |
|     15 | ALIEN CENTER   |         72 | 2005-08-20 15:11:48 |
|     15 | ALIEN CENTER   |         73 | 2005-07-06 15:51:58 |
|     15 | ALIEN CENTER   |         73 | 2005-07-30 14:48:24 |
|     15 | ALIEN CENTER   |         73 | 2005-08-20 22:32:11 |
|     15 | ALIEN CENTER   |         74 | 2005-07-27 00:15:18 |
|     15 | ALIEN CENTER   |         74 | 2005-08-23 19:21:22 |
|     15 | ALIEN CENTER   |         75 | 2005-07-09 02:58:41 |
|     15 | ALIEN CENTER   |         75 | 2005-07-29 23:52:01 |
|     15 | ALIEN CENTER   |         75 | 2005-08-18 21:55:01 |
|     15 | ALIEN CENTER   |         76 | 2005-06-15 08:01:29 |
|     15 | ALIEN CENTER   |         76 | 2005-07-07 18:31:50 |
|     15 | ALIEN CENTER   |         76 | 2005-08-01 01:49:36 |
|     15 | ALIEN CENTER   |         76 | 2005-08-17 07:26:47 |
+--------+----------------+------------+---------------------+
32 rows in set (0.01 sec)
```

The results include all rentals of all films in inventory, but the film *Alice Fantasia* has `null` values for the columns from both outer-joined tables.

Cross Joins

Back in Chapter 5, I introduced the concept of a Cartesian product, which is essentially the result of joining multiple tables without specifying any join conditions. Cartesian products are used fairly frequently by accident (e.g., forgetting to add the join condition

to the `from` clause) but are not so common otherwise. If, however, you *do* intend to generate the Cartesian product of two tables, you should specify a *cross join*, as in:

```
mysql> SELECT c.name category_name, l.name language_name
    -> FROM category c
    ->   CROSS JOIN language l;
+---------------+---------------+
| category_name | language_name |
+---------------+---------------+
| Action        | English       |
| Action        | Italian       |
| Action        | Japanese      |
| Action        | Mandarin      |
| Action        | French        |
| Action        | German        |
| Animation     | English       |
| Animation     | Italian       |
| Animation     | Japanese      |
| Animation     | Mandarin      |
| Animation     | French        |
| Animation     | German        |
...
| Sports        | English       |
| Sports        | Italian       |
| Sports        | Japanese      |
| Sports        | Mandarin      |
| Sports        | French        |
| Sports        | German        |
| Travel        | English       |
| Travel        | Italian       |
| Travel        | Japanese      |
| Travel        | Mandarin      |
| Travel        | French        |
| Travel        | German        |
+---------------+---------------+
96 rows in set (0.00 sec)
```

This query generates the Cartesian product of the `category` and `language` tables, resulting in 96 rows (16 `category` rows × 6 `language` rows). But now that you know what a cross join is and how to specify it, what is it used for? Most SQL books will describe what a cross join is and then tell you that it is seldom useful, but I would like to share with you a situation in which I find the cross join to be quite helpful.

In Chapter 9, I discussed how to use subqueries to fabricate tables. The example I used showed how to build a three-row table that could be joined to other tables. Here's the fabricated table from the example:

```
mysql> SELECT 'Small Fry' name, 0 low_limit, 74.99 high_limit
    -> UNION ALL
    -> SELECT 'Average Joes' name, 75 low_limit, 149.99 high_limit
    -> UNION ALL
    -> SELECT 'Heavy Hitters' name, 150 low_limit, 9999999.99 high_limit;
+--------------+-----------+------------+
| name         | low_limit | high_limit |
+--------------+-----------+------------+
| Small Fry    |         0 |      74.99 |
| Average Joes |        75 |     149.99 |
| Heavy Hitters |      150 | 9999999.99 |
+--------------+-----------+------------+
3 rows in set (0.00 sec)
```

While this table was exactly what was needed for placing customers into three groups based on their total film payments, this strategy of merging single-row tables using the set operator `union all` doesn't work very well if you need to fabricate a large table.

Say, for example, that you want to create a query that generates a row for every day in the year 2020 but you don't have a table in your database that contains a row for every day. Using the strategy from the example in Chapter 9, you could do something like the following:

```
SELECT '2020-01-01' dt
UNION ALL
SELECT '2020-01-02' dt
UNION ALL
SELECT '2020-01-03' dt
UNION ALL
...
...
...
SELECT '2020-12-29' dt
UNION ALL
SELECT '2020-12-30' dt
UNION ALL
SELECT '2020-12-31' dt
```

Building a query that merges together the results of 366 queries is a bit tedious, so maybe a different strategy is needed. What if you generate a table with 366 rows (2020 is a leap year) with a single column containing a number between 0 and 366 and then add that number of days to January 1, 2020? Here's one possible method to generate such a table:

```
mysql> SELECT ones.num + tens.num + hundreds.num
    -> FROM
    -> (SELECT 0 num UNION ALL
    -> SELECT 1 num UNION ALL
    -> SELECT 2 num UNION ALL
    -> SELECT 3 num UNION ALL
    -> SELECT 4 num UNION ALL
    -> SELECT 5 num UNION ALL
    -> SELECT 6 num UNION ALL
    -> SELECT 7 num UNION ALL
    -> SELECT 8 num UNION ALL
    -> SELECT 9 num) ones
    -> CROSS JOIN
    -> (SELECT 0 num UNION ALL
    -> SELECT 10 num UNION ALL
    -> SELECT 20 num UNION ALL
    -> SELECT 30 num UNION ALL
    -> SELECT 40 num UNION ALL
    -> SELECT 50 num UNION ALL
    -> SELECT 60 num UNION ALL
    -> SELECT 70 num UNION ALL
    -> SELECT 80 num UNION ALL
    -> SELECT 90 num) tens
    -> CROSS JOIN
    -> (SELECT 0 num UNION ALL
    -> SELECT 100 num UNION ALL
    -> SELECT 200 num UNION ALL
    -> SELECT 300 num) hundreds;
+-----------------------------------+
| ones.num + tens.num + hundreds.num |
+-----------------------------------+
|                                 0 |
|                                 1 |
|                                 2 |
|                                 3 |
|                                 4 |
|                                 5 |
|                                 6 |
|                                 7 |
```

```
|                   8 |
|                   9 |
|                  10 |
|                  11 |
|                  12 |
...
...
...
|                 391 |
|                 392 |
|                 393 |
|                 394 |
|                 395 |
|                 396 |
|                 397 |
|                 398 |
|                 399 |
+---------------------------------+
400 rows in set (0.00 sec)
```

If you take the Cartesian product of the three sets {0, 1, 2, 3, 4, 5, 6, 7, 8, 9}, {0, 10, 20, 30, 40, 50, 60, 70, 80, 90}, and {0, 100, 200, 300} and add the values in the three columns, you get a 400-row result set containing all numbers between 0 and 399. While this is more than the 366 rows needed to generate the set of days in 2020, it's easy enough to get rid of the excess rows, and I'll show you how shortly.

The next step is to convert the set of numbers to a set of dates. To do this, I will use the `date_add()` function to add each number in the result set to January 1, 2020. Then I'll add a filter condition to throw away any dates that venture into 2021:

```
mysql> SELECT DATE_ADD('2020-01-01',
    ->   INTERVAL (ones.num + tens.num + hundreds.num) DAY) dt
    -> FROM
    ->  (SELECT 0 num UNION ALL
    ->   SELECT 1 num UNION ALL
    ->   SELECT 2 num UNION ALL
    ->   SELECT 3 num UNION ALL
    ->   SELECT 4 num UNION ALL
    ->   SELECT 5 num UNION ALL
    ->   SELECT 6 num UNION ALL
    ->   SELECT 7 num UNION ALL
    ->   SELECT 8 num UNION ALL
    ->   SELECT 9 num) ones
    ->   CROSS JOIN
```

```
    ->  (SELECT 0 num UNION ALL
    ->   SELECT 10 num UNION ALL
    ->   SELECT 20 num UNION ALL
    ->   SELECT 30 num UNION ALL
    ->   SELECT 40 num UNION ALL
    ->   SELECT 50 num UNION ALL
    ->   SELECT 60 num UNION ALL
    ->   SELECT 70 num UNION ALL
    ->   SELECT 80 num UNION ALL
    ->   SELECT 90 num) tens
    ->   CROSS JOIN
    ->  (SELECT 0 num UNION ALL
    ->   SELECT 100 num UNION ALL
    ->   SELECT 200 num UNION ALL
    ->   SELECT 300 num) hundreds
    -> WHERE DATE_ADD('2020-01-01',
    ->   INTERVAL (ones.num + tens.num + hundreds.num) DAY) < '2021-01-01'
    -> ORDER BY 1;
+------------+
| dt         |
+------------+
| 2020-01-01 |
| 2020-01-02 |
| 2020-01-03 |
| 2020-01-04 |
| 2020-01-05 |
| 2020-01-06 |
| 2020-01-07 |
| 2020-01-08 |
...
...
...
| 2020-02-26 |
| 2020-02-27 |
| 2020-02-28 |
| 2020-02-29 |
| 2020-03-01 |
| 2020-03-02 |
| 2020-03-03 |
...
...
...
| 2020-12-24 |
| 2020-12-25 |
| 2020-12-26 |
| 2020-12-27 |
```

```
| 2020-12-28 |
| 2020-12-29 |
| 2020-12-30 |
| 2020-12-31 |
+------------+
366 rows in set (0.03 sec)
```

The nice thing about this approach is that the result set automatically includes the extra leap day (February 29) without your intervention, since the database server figures it out when it adds 59 days to January 1, 2020.

Now that you have a mechanism for fabricating all the days in 2020, what should you do with it? Well, you might be asked to generate a report that shows every day in 2020 along with the number of film rentals on that day. The report needs to include every day of the year, including days when no films are rented. Here's what the query might look like (using the year 2005 to match the data in the `rental` table):

```
mysql> SELECT days.dt, COUNT(r.rental_id) num_rentals
    -> FROM rental r
    ->   RIGHT OUTER JOIN
    -> (SELECT DATE_ADD('2005-01-01',
    ->    INTERVAL (ones.num + tens.num + hundreds.num) DAY) dt
    ->   FROM
    ->    (SELECT 0 num UNION ALL
    ->     SELECT 1 num UNION ALL
    ->     SELECT 2 num UNION ALL
    ->     SELECT 3 num UNION ALL
    ->     SELECT 4 num UNION ALL
    ->     SELECT 5 num UNION ALL
    ->     SELECT 6 num UNION ALL
    ->     SELECT 7 num UNION ALL
    ->     SELECT 8 num UNION ALL
    ->     SELECT 9 num) ones
    ->     CROSS JOIN
    ->    (SELECT 0 num UNION ALL
    ->     SELECT 10 num UNION ALL
    ->     SELECT 20 num UNION ALL
    ->     SELECT 30 num UNION ALL
    ->     SELECT 40 num UNION ALL
    ->     SELECT 50 num UNION ALL
    ->     SELECT 60 num UNION ALL
    ->     SELECT 70 num UNION ALL
    ->     SELECT 80 num UNION ALL
    ->     SELECT 90 num) tens
```

```
    ->     CROSS JOIN
    ->    (SELECT 0 num UNION ALL
    ->     SELECT 100 num UNION ALL
    ->     SELECT 200 num UNION ALL
    ->     SELECT 300 num) hundreds
    ->    WHERE DATE_ADD('2005-01-01',
    ->     INTERVAL (ones.num + tens.num + hundreds.num) DAY)
    ->       < '2006-01-01'
    ->  ) days
    ->   ON days.dt = date(r.rental_date)
    -> GROUP BY days.dt
    -> ORDER BY 1;
+------------+-------------+
| dt         | num_rentals |
+------------+-------------+
| 2005-01-01 |           0 |
| 2005-01-02 |           0 |
| 2005-01-03 |           0 |
| 2005-01-04 |           0 |
...
| 2005-05-23 |           0 |
| 2005-05-24 |           8 |
| 2005-05-25 |         137 |
| 2005-05-26 |         174 |
| 2005-05-27 |         166 |
| 2005-05-28 |         196 |
| 2005-05-29 |         154 |
| 2005-05-30 |         158 |
| 2005-05-31 |         163 |
| 2005-06-01 |           0 |
...
| 2005-06-13 |           0 |
| 2005-06-14 |          16 |
| 2005-06-15 |         348 |
| 2005-06-16 |         324 |
| 2005-06-17 |         325 |
| 2005-06-18 |         344 |
| 2005-06-19 |         348 |
| 2005-06-20 |         331 |
| 2005-06-21 |         275 |
| 2005-06-22 |           0 |
...
| 2005-12-27 |           0 |
| 2005-12-28 |           0 |
| 2005-12-29 |           0 |
| 2005-12-30 |           0 |
```

```
| 2005-12-31 |          0 |
+------------+------------+
365 rows in set (8.99 sec)
```

This is one of the more interesting queries thus far in the book, in that it includes cross joins, outer joins, a date function, grouping, set operations (`union all`), and an aggregate function (`count()`). It is also not the most elegant solution to the given problem, but it should serve as an example of how, with a little creativity and a firm grasp on the language, you can make even a seldom-used feature like cross joins a potent tool in your SQL toolkit.

Natural Joins

If you are lazy (and aren't we all), you can choose a join type that allows you to name the tables to be joined but lets the database server determine what the join conditions need to be. Known as the *natural join*, this join type relies on identical column names across multiple tables to infer the proper join conditions. For example, the `rental` table includes a column named `customer_id` , which is the foreign key to the `customer` table, whose primary key is also named `customer_id` . Thus, you could try to write a query that uses `natural join` to join the two tables:

```
mysql> SELECT c.first_name, c.last_name, date(r.rental_date)
    -> FROM customer c
    ->   NATURAL JOIN rental r;
Empty set (0.04 sec)
```

Because you specified a natural join, the server inspected the table definitions and added the join condition `r.customer_id = c.customer_id` to join the two tables. This would have worked fine, but in the Sakila schema all of the tables include the column `last_update` to show when each row was last modified, so the server is also adding the join condition `r.last_update = c.last_update` , which causes the query to return no data.

The only way around this issue is to use a subquery to restrict the columns for at least one of the tables:

```
mysql> SELECT cust.first_name, cust.last_name, date(r.rental_date)
    -> FROM
    ->  (SELECT customer_id, first_name, last_name
    ->   FROM customer
```

```
  -> ) cust
  -> NATURAL JOIN rental r;
+------------+-----------+--------------------+
| first_name | last_name | date(r.rental_date) |
+------------+-----------+--------------------+
| MARY       | SMITH     | 2005-05-25         |
| MARY       | SMITH     | 2005-05-28         |
| MARY       | SMITH     | 2005-06-15         |
| MARY       | SMITH     | 2005-06-15         |
| MARY       | SMITH     | 2005-06-15         |
| MARY       | SMITH     | 2005-06-16         |
| MARY       | SMITH     | 2005-06-18         |
| MARY       | SMITH     | 2005-06-18         |
...
| AUSTIN     | CINTRON   | 2005-08-21         |
| AUSTIN     | CINTRON   | 2005-08-21         |
| AUSTIN     | CINTRON   | 2005-08-21         |
| AUSTIN     | CINTRON   | 2005-08-23         |
| AUSTIN     | CINTRON   | 2005-08-23         |
| AUSTIN     | CINTRON   | 2005-08-23         |
+------------+-----------+--------------------+
16044 rows in set (0.03 sec)
```

So, is the reduced wear and tear on the old fingers from not having to type the join condition worth the trouble? Absolutely not; you should avoid this join type and use inner joins with explicit join conditions.

Test Your Knowledge

The following exercises test your understanding of outer and cross joins. Please see Appendix B for solutions.

Exercise 10-1

Using the following table definitions and data, write a query that returns each customer name along with their total payments:

```
                Customer:
Customer_id     Name
-----------     ---------------
1               John Smith
2               Kathy Jones
```

```
3          Greg Oliver

                    Payment:
Payment_id   Customer_id  Amount
----------   -----------  --------
101          1            8.99
102          3            4.99
103          1            7.99
```

Include all customers, even if no payment records exist for that customer.

Exercise 10-2

Reformulate your query from Exercise 10-1 to use the other outer join type (e.g., if you used a left outer join in Exercise 10-1, use a right outer join this time) such that the results are identical to Exercise 10-1.

Exercise 10-3 (Extra Credit)

Devise a query that will generate the set {1, 2, 3, ..., 99, 100}. (Hint: use a cross join with at least two `from` clause subqueries.)

Previous chapter
< 9. Subqueries

Next chapter
11. Conditional Logic >